Doc. No.: WG21/P0735R1
Date: 2019-06-17
Reply-to: Will Deacon, Jade Alglave
Email: will.deacon@arm.com, jade.alglave@arm.com
Authors: Will Deacon and Jade Alglave with input from Olivier Giroux and Paul McKenney
Audience: CWG

Changelog

P0735R1: Add additional author/reply-to. Add changelog. Update to account for ensuing revisions to the IS.
D0735R1: Update to account for ensuing revisions to the IS *(Kona)*
D0735R0: Initial proposal to SG1 *(Albuquerque)*

P0735R1: Interaction of `memory_order_consume` with release sequences

The current definition of `memory_order_consume` is not sufficient to allow implementations to map a consume load to a "plain" load instruction (i.e. without using fences), despite this being the intention of the original proposal. Instead, `memory_order_consume` is typically treated as though the program had specified `memory_order_acquire`, which is now preferred by the standard (31.4p1.3 [atomics.order], P0371).

Work is ongoing to make `memory_order_consume` viable for implementations (P0462, P0190), but its interaction with release sequences remains unchanged and continues to be problematic for ARMv8 and potentially future architectures.

Release sequences

Release sequences provide a way to extend order from a release operation to other stores to the same object that are adjacent in the modification order. Consequently, an acquire operation can establish a "synchronizes with" relation with a release store by reading from any member of the release sequence headed by that store (31.4p2 [atomics.order]). An example use-case for release sequences is when a locking implementation places other flags into the lock word, which can be modified using relaxed read-modify-write operations by threads that do not hold the lock. Without the ordering guarantees of the release sequence, the read-modify-write operations updating the flags would need to use `memory_order_acq_rel` to ensure that lock operations synchronize with prior unlock operations for a given lock.

Consume operations interact with release sequences in a similar manner to acquire operations via the "dependency-ordered before" relation (6.8.2.1p8 [intro.races]). One notable difference between the behaviour of consume and acquire operations in this regard is that the consume operation must be performed by a different thread than the one performing the release operation.

The following example shows a release store that is dependency-ordered before a relaxed load, due to the release sequence from B to C:

```
int x, y;
atomic<int *> datap;

void p0(void)
{
        x = 42; /* A */
        datap.store(&y, memory_order_release); /* B */
}

void p1(void)
{
        int *p, *q, r;

        do {
                p = datap.exchange(&x, memory_order_relaxed); /* C */
        } while (p != &y);

        q = datap.load(memory_order_consume); /* D */
        r = *q; /* E */
}
```

The C++ memory model establishes the following relations, which can be used to construct "happens before" for the program.

- A is sequenced-before B
  - therefore A happens before B
- C is in the release sequence headed by B
- D reads from C
  - therefore B is dependency-ordered before D
- D carries a dependency to E
  - therefore B is dependency-ordered before E,
  - B inter-thread happens before E, and
  - B happens before E

The "happens before" relation requires that `r == 42`, however this is not guaranteed by the ARMv8 architecture if the existing compiler mappings are changed to map `memory_order_consume` loads to `LDR` (the same as `memory_order_relaxed`). There is production silicon capable of exhibiting this behaviour.

Behaviour on hardware

In the previous example, P1 can be compiled to the following AArch64 instructions:

```
/*
 * X0 = &x
```

```
 * X1 = &y
 * X2 = &datap
 * X3 = p
 * X4 = q
 * W5 = r
 */
.L1:    SWP     X3, X0, [X2]    // exchange
        CMP     X3, X1
        B.NE    .L1
        LDR     X4, [X2]        // consume load
        LDR     W5, [X4]        // dependent load
```

In the absence of any fence instructions, the CPU can forward the write to `datap` from the `SWP` instruction to the `LDR` of the consume load, speculating past the conditional branch. The dependent load can then complete before the `SWP` has returned data, returning a stale value for `x` (i.e. not 42).

This behaviour is permitted by the ARMv8 memory model and is likely to be permitted on other upcoming architectures.

Possible solutions

While it is tempting to fix this problem by changing "dependency-ordered before" to require that the consume load must read a data value written by another thread, this does not resolve the problem for non-multi-copy atomic architectures that can perform forwarding between threads using a shared pre-cache store buffer.

Another option is to restrict the read-modify-write operations that can appear in a release sequence used to establish a "dependency-ordered before" relation to those with implicit data dependencies (e.g. `atomic_fetch_*`). This would notably omit `compare_exchange` operations which only provide a control dependency and are not sufficent to order against subsequent loads. Since `compare_exchange` is often used to implement `atomic_fetch_*` operations, then ordering may be broken in certain corner-cases (e.g. saturating arithmetic).

Given that there appear to be no known use-cases for release sequences in conjunction with `memory_order_consume` operations, this paper instead proposes to remove then entirely from the definition of "dependency-ordered before".

Proposed wording

Change 6.8.2.1p8 [intro.races] to remove release sequences from the definition of "dependency-ordered before":

> An evaluation A is *dependency-ordered before* an evaluation B if
>
> - A performs a release operation on an atomic object M, and, in another thread, B performs a consume operation on M and reads ~~a value written by any side effect in the release sequence headed~~ the value written by A, or
> - for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.

[*Note:* The relation "is dependency-ordered before" is analogous to "synchronizes with", but uses release/consume in place of release/acquire. — *end note*]